

Для начала нужно зайти в директорию библиотеки zlib и сконфигурировать makefile.

```
wild4est | wild4est-laptop | ~/Desktop/MyFavoriteFolder/learn/Eltex/LinuxKernel/zlib-1.3.2 $ ./configure --prefix=$PWD/_install --static
Checking for gcc...
Building static library libz.a version 1.3.2 with gcc.
Checking for size_t... Yes.
Checking for off64_t... Yes.
Checking for fseeko... Yes.
Checking for strerror... Yes.
Checking for unistd.h... Yes.
Checking for stdarg.h... Yes.
Checking whether to use vs[n]printf() or s[n]printf()... using vs[n]printf().
Checking for vsnprintf() in stdio.h... Yes.
Checking for return value of vsnprintf()... Yes.
Checking for attribute(visibility) support... Yes.
Checking for s390x build ... No.
```

Рисунок 1 — конфигурация makefile для zlib

После нужно вызвать команду make, подставив при этом в переменную CC нужный компилятор, и make install.

```
wild4est | wild4est-laptop | ~/Desktop/MyFavoriteFolder/learn/Eltex/LinuxKernel/zlib-1.3.2 $ make CC=arm-linux-gnueabi-gcc
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -I. -c -o example.o test/example.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o adler32.o adler32.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o crc32.o crc32.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o deflate.o deflate.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o infback.o infback.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o inffast.o inffast.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o inflate.o inflate.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o inftrees.o inftrees.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o trees.o trees.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o zutil.o zutil.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o compress.o compress.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o uncompress.o uncompress.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o gzclose.o gzclose.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o gzlib.o gzlib.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o gzread.o gzread.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o gzwrite.o gzwrite.c
ar rc libz.a adler32.o crc32.o deflate.o infback.o inffast.o inflate.o inftrees.o trees.o zutil.o compress.o uncompress.o gzclose
.o gzlib.o gzread.o gzwrite.o
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -o example example.o -L. libz.a
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -I. -c -o minigzip.o test/minigzip.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -o minigzip minigzip.o -L. libz.a
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -I. -D_FILE_OFFSET_BITS=64 -c -o example64.o test/exam
ple.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -o example64 example64.o -L. libz.a
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -I. -D_FILE_OFFSET_BITS=64 -c -o minigzip64.o test/min
igzip.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -o minigzip64 minigzip64.o -L. libz.a
```

Рисунок 2 — выполнение команды make для zlib

Теперь можно попробовать собрать makefile для openssl и убедиться, что zlib собрана корректно.

```
wild4est | wild4est-laptop | ~/Desktop/MyFavoriteFolder/learn/Eltex/LinuxKernel/openssl-portable-V_10_3_P1 $ ./configure --h
ost=arm-linux-gnueabi --with-zlib="$PWD/../zlib/_install" --prefix="$PWD/_install"
checking for arm-linux-gnueabi-gcc... no
checking for arm-linux-gnueabi-gcc... arm-linux-gnueabi-gcc
checking whether the C compiler works... yes
checking for C compiler default output file name... a.out
checking for suffix of executables...
checking whether we are cross compiling... yes
checking for suffix of object files... o
checking whether the compiler supports GNU C... yes
```

Рисунок 3 — конфигурация makefile для openssl

В ходе сборки возникнет ошибка об отсутствии openssl, что означает zlib собрана корректно.

```
checking whether snprintf can declare const char *fmt... yes
checking whether system supports SO_PEERCRED getsockopt... yes
checking if openpty correctly handles controlling tty... cross-compiling, assuming yes
checking whether AI_NUMERICSERV is declared... yes
checking if SA_RESTART signals interrupt select()... configure: WARNING: cross compiling: assuming yes
checking for getpgrp... yes
checking if getpgrp accepts zero args... yes
checking for openssl... /usr/bin/openssl
configure: error: *** working libcrypto not found, check config.log
```

Рисунок 4 — ошибка об отсутствии openssl

Теперь необходимо собрать саму openssl. Для этого заходим в папку проекта и открываем файл Configure. В данном файле можно найти опции сборки. Среди них можно найти опции «--prefix» для указания пути сборки и «--cross-compile-prefix» для указания префикса кросс-компиляции.

```
# Options:
#
# --config      add the given configuration file, which will be read after
#               any "Configurations*" files that are found in the same
#               directory as this script.
# --prefix      prefix for the OpenSSL installation, which includes the
#               directories bin, lib, include, share/man, share/doc/openssl
#               This becomes the value of INSTALLTOP in Makefile
#               (Default: /usr/local)
# --openssldir  OpenSSL data area, such as openssl.cnf, certificates and keys.
#               If it's a relative directory, it will be added on the directory
#               given with --prefix.
#               This becomes the value of OPENSSLDIR in Makefile and in C.
#               (Default: PREFIX/ssl)
# --banner=".." Output specified text instead of default completion banner
#
# -w           Don't wait after showing a Configure warning
#
# --cross-compile-prefix Add specified prefix to binutils components.
#
```

Рисунок 5 — опции конфигурации

Конфигурируем makefile с использованием данных опций.

```
wild4est | wild4est-laptop | ~/Desktop/MyFavoriteFolder/learn/Eltex/LinuxKernel/openssl-4.0.0 $ ./Configure --prefix="$PWD/_
install" --cross-compile-prefix=arm-linux-gnueabi-
Configuring OpenSSL version 4.0.0 for target linux-x86_64
Using os-specific seed configuration
Created configdata.pm
Running configdata.pm
Created Makefile.in
Created Makefile
Created include/openssl/configuration.h

*****
***      OpenSSL has been successfully configured      ***
***      ***
***      If you encounter a problem while building, please open an ***
***      issue on GitHub <https://github.com/openssl/openssl/issues> ***
***      and include the output from the following command: ***
***      ***
***      perl configdata.pm --dump ***
***      ***
***      (If you are new to OpenSSL, you might want to consult the ***
***      'Troubleshooting' section in the INSTALL.md file first) ***
***      ***
*****
```

Рисунок 6 — конфигурация makefile для openssl

Теперь можно зайти в сам makefile и убедиться, что нужные параметры были подставлены.

```
3868 # We let the C compiler driver to take care of .s files. This is done in
3869 # order to be excused from maintaining a separate set of architecture
3870 # dependent assembler flags. E.g. if you throw -mcpu=ultrasparc at SPARC
3871 # gcc, then the driver will automatically translate it to -xarch=v8plus
3872 # and pass it down to assembler. In any case, we do not define AS or
3873 # ASFLAGS for this reason.
3874
3875 CROSS_COMPILE=arm-linux-gnueabi-
3876 CC=$(CROSS_COMPILE)gcc
3877 CXX=$(CROSS_COMPILE)g++
3878 CPPFLAGS=
3879 CFLAGS=-Wall -O3
3880 CXXFLAGS=-Wall -O3
3881 LDFLAGS=
3882 EX_LIBS=
3883 OBJCOPY=objcopy
```

Рисунок 7 — сконфигурированный makefile для openssl

Однако при выполнении команды make возникнет ошибка.

```
wild4est | wild4est-laptop | ~/Desktop/MyFavoriteFolder/learn/Eltex/LinuxKernel/openssl-4.0.0 $ make
/usr/bin/perl util/mkinstallvars.pl COMMENT="This file should be used when building against this OpenSSL build, and should never be installed" PREFIX=. BINDIR=apps APPLINKDIR=ms LIBDIR= INCLUDEDIR=include "INCLUDEDIR=../include" MODULESDIR=providers libdir= CMAKECONFIGDIR= PKGCONFIGDIR= "VERSION=4.0.0" "LDLIBS=-ldl -pthread" > builddata.pm
DEBUG: all keys: APPLINKDIR, BINDIR, CMAKECONFIGDIR, COMMENT, INCLUDEDIR, LDLIBS, LIBDIR, MODULESDIR, PKGCONFIGDIR, PREFIX, VERSION, libdir
DEBUG: LIBDIR = , libdir = => LIBDIR =
DEBUG: PREFIX = . => PREFIX = /home/wild4est/Desktop/MyFavoriteFolder/learn/Eltex/LinuxKernel/openssl-4.0.0
DEBUG: libdir = => libdir = /home/wild4est/Desktop/MyFavoriteFolder/learn/Eltex/LinuxKernel/openssl-4.0.0
DEBUG: BINDIR = apps => BINDIR = /home/wild4est/Desktop/MyFavoriteFolder/learn/Eltex/LinuxKernel/openssl-4.0.0/apps, BINDIR_REL_PREFIX = apps
DEBUG: LIBDIR = => LIBDIR = /home/wild4est/Desktop/MyFavoriteFolder/learn/Eltex/LinuxKernel/openssl-4.0.0, LIBDIR_REL_PREFIX =
DEBUG: INCLUDEDIR = [ include, ./include ] => INCLUDEDIR = [ /home/wild4est/Desktop/MyFavoriteFolder/learn/Eltex/LinuxKernel/openssl-4.0.0/include, /home/wild4est/Desktop/MyFavoriteFolder/learn/Eltex/LinuxKernel/openssl-4.0.0/include ], INCLUDEDIR_REL_PREFIX = [ include, ./include ]
DEBUG: APPLINKDIR = ms => APPLINKDIR = /home/wild4est/Desktop/MyFavoriteFolder/learn/Eltex/LinuxKernel/openssl-4.0.0/ms, APPLINKDIR_REL_PREFIX = ms
make[1]: depend && "make" _build_sw
make[1]: Entering directory '/home/wild4est/Desktop/MyFavoriteFolder/learn/Eltex/LinuxKernel/openssl-4.0.0'
make[1]: Leaving directory '/home/wild4est/Desktop/MyFavoriteFolder/learn/Eltex/LinuxKernel/openssl-4.0.0'
make[1]: Entering directory '/home/wild4est/Desktop/MyFavoriteFolder/learn/Eltex/LinuxKernel/openssl-4.0.0'
arm-linux-gnueabi-gcc -I. -Iinclude -Iapps/include -fPIC -pthread -m64 -Wa,--noexecstack -Wall -O3 -DL_ENDIAN -DOPENSSL_PIC -DOPENSSLDIR=""/home/wild4est/Desktop/MyFavoriteFolder/learn/Eltex/LinuxKernel/openssl-4.0.0/_install/ssl/" -DMODULESDIR=""/home/wild4est/Desktop/MyFavoriteFolder/learn/Eltex/LinuxKernel/openssl-4.0.0/_install/lib64/openssl-modules/" -DNDEBUG -MM -MF apps/lib/libapps-lib-app_libctx.d.tmp -c -o apps/lib/libapps-lib-app_libctx.o apps/lib/app_libctx.c
arm-linux-gnueabi-gcc: error: unrecognized command-line option '-m64'
make[1]: *** [Makefile:6124: apps/lib/libapps-lib-app_libctx.o] Error 1
make[1]: Leaving directory '/home/wild4est/Desktop/MyFavoriteFolder/learn/Eltex/LinuxKernel/openssl-4.0.0'
```

Рисунок 8-9 выполнение команды make

Данная ошибка, вызвана тем, что makefile пытается собрать openssl под 64-битную архитектуру в то время, как используется 32-битный компилятор.

```
3909 CNF_CPPFLAGS=-DNDEBUG
3910 CNF_CFLAGS=-pthread -m64 -Wa,--noexecstack
3911 CNF_CXXFLAGS=-std=c++11 -pthread -m64
3912 CNF_LDFLAGS=
3913 CNF_EX_LIBS=-ldl -pthread
3914
```

Рисунок 10 — сконфигурированный makefile для openssl

Значит во время выполнения конфигурации нужно указать подходящую архитектуру. Но для начала нужно её найти. Для этого вызываем команду ./Configure LIST. И затем в перечне можно найти linux-generic32.

```
wild4est | wild4est-laptop | ~/Desktop/MyFavoriteFolder/learn/Eltex/LinuxKernel/openssl-4.0.0 $ ./Configure LIST
BC-32
BC-64
BS2000-OSD
BSD-aarch64
BSD-armv4
BSD-generic32
BSD-generic64
BSD-ia64
BSD-nofdef-generic32
BSD-nofdef-generic64
BSD-nofdef-ia64
BSD-nofdef-sparc64
BSD-nofdef-sparcv8
BSD-nofdef-x86
BSD-nofdef-x86_elf
BSD-nofdef-x86_64
BSD-ppc

linux-armv4
linux-c64xplus
linux-elf
linux-generic32
linux-generic64
linux-ia64
```

Рисунок 11-12 — список поддерживаемых архитектур

Теперь можно ещё раз выполнить конфигурацию с указанием нужной архитектуры.

```
wild4est | wild4est-laptop | ~/Desktop/MyFavoriteFolder/learn/Eltex/LinuxKernel/openssl-4.0.0 $ ./Configure --prefix="$PWD/_install" --cross-compile-prefix=arm-linux-gnueabihf- linux-generic32
Configuring OpenSSL version 4.0.0 for target linux-generic32
Using os-specific seed configuration
Created configdata.pm
Running configdata.pm
Created Makefile.in
Created Makefile

*****
***
***   OpenSSL has been successfully configured
***
***   If you encounter a problem while building, please open an
***   issue on GitHub <https://github.com/openssl/openssl/issues>
***   and include the output from the following command:
***
***       perl configdata.pm --dump
***
***   (If you are new to OpenSSL, you might want to consult the
***   'Troubleshooting' section in the INSTALL.md file first)
***
*****
```

Рисунок 13 — конфигурация makefile для openssl

Можно ещё раз зайти в makefile и убедиться, что теперь в нём указаны верные параметры сборки.

```
3868 CROSS_COMPILE=arm-linux-gnueabihf-
3869 CC=$(CROSS_COMPILE)gcc
3870 CXX=$(CROSS_COMPILE)g++
3871 CPPFLAGS=
3872 CFLAGS=-Wall -O3
3873 CXXFLAGS=-Wall -O3
3874 LDFLAGS=
3875 EX_LIBS=
3876 OBJCOPY=objcopy
3877

3902 CNF_CPPFLAGS=-DNDEBUG
3903 CNF_CFLAGS=-pthread
3904 CNF_CXXFLAGS=-std=c++11 -pthread
3905 CNF_LDFLAGS=
3906 CNF_EX_LIBS=-ldl -pthread
3907
```

Рисунок 14 — сконфигурированный makefile для openssl

Теперь после выполнения команды make и make install нету никаких ошибок.

Можно переходить к конфигурации openssl.

```
Report bugs to <openssh-unix-dev@mindrot.org>.
wild4est | wild4est-laptop | ~/Desktop/MyFavoriteFolder/learn/Eltex/LinuxKernel/openssh-portable-V_10_3_P1 $ ./configure --host=arm-linux-gnueabihf --with-zlib="$PWD/../zlib/_install" --with-ssl-dir="$PWD/../openssl/_install" --prefix="$PWD/_install"
```

Рисунок 15 — конфигурация makefile для openssl

После формирования makefile можно выполнить команду make и make install-nosysconf.

```
install: $(CONFIGFILES) $(MANPAGES) $(TARGETS) install-files install-sysconf host-key check-config
install-nokeys: $(CONFIGFILES) $(MANPAGES) $(TARGETS) install-files install-sysconf
install-nosysconf: $(CONFIGFILES) $(MANPAGES) $(TARGETS) install-files
```

Рисунок 16 — makefile для openssl

После успешной установки все файлы из папок «/zlib/\_install/lib», «/openssl/\_install/lib» и «/openssh/\_install» необходимо добавить в ранее созданную файловую систему в budybox. Так же в папку lib необходимо скопировать файлы «ld-linux-armhf.so.3», «libc.so» и «libc.so.6» из директории «/usr/arm-linux-gnueabi/lib».

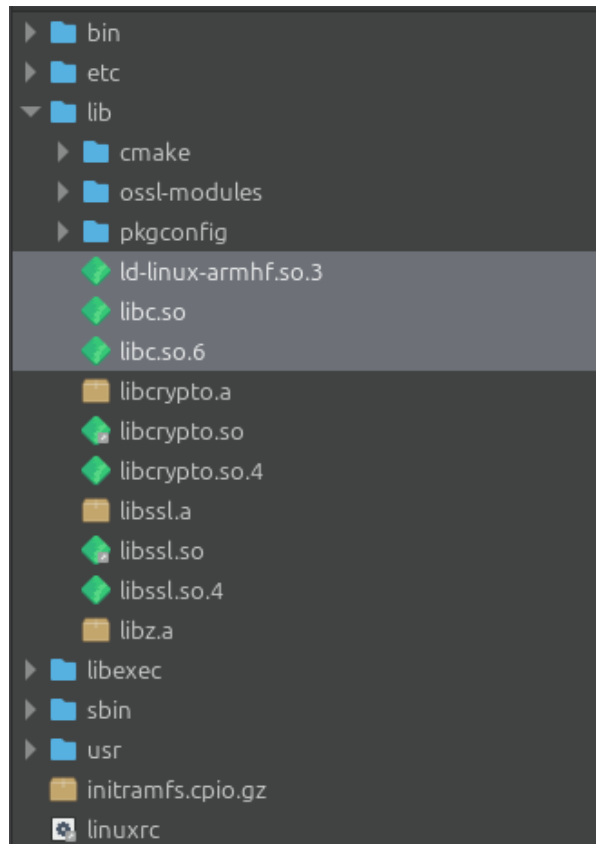


Рисунок 17 — корневая файловая система прошивки

Теперь когда все необходимые файлы находятся в файловой системе, файловую систему можно заархивировать и запустить qemu.

```
wild4est | wild4est-laptop | ~/Desktop/MyFavoriteFolder/learn/Eltex/LinuxKernel/tmp $ QEMU_AUDIO_DRV=none qemu-system-arm -M vexpress-a9 -kernel zImage -initrd initramfs.cpio.gz -dtb vexpress-v2p-ca9.dtb -append "console=ttyAMA0 rdinit=/bin/ash" -nographic
[ 0.000000] Booting Linux on physical CPU 0x0
[ 0.000000] Linux version 7.0.11 (wild4est@wild4est-laptop) (arm-linux-gnueabi-gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0, GNU ld (GNU Binutils for Ubuntu) 2.42) #1 SMP Sun Jun  7 14:26:02 +07 2026
[ 0.000000] CPU: ARMv7 Processor [410fc090] revision 0 (ARMv7), cr=10c5387d
[ 0.000000] CPU: PIPT / VIPT nonaliasing data cache, VIPT nonaliasing instruction cache
```

Рисунок 18 — запуск образа ядра

При попытке вызвать команду ssh возникнет ошибка, связанная с отсутствием файла «/dev/null».

```
~ # ls
bin      etc      libexec  root     usr
dev      lib      linuxrc  sbin
~ # ssh
Couldn't open /dev/null: No such file or directory
~ #
```

Рисунок 19 — ошибка отсутствия файла /dev/null

Попробуем данный файл создать.

```
~ # mknod /dev/null c 1 3
~ # chmod 666 /dev/null
~ # cd dev
/dev # ls
console null
/dev #
```

Рисунок 20 — создание файла /dev/null

Теперь при попытке запустить команду ssh возникает ошибка, связанная с отсутствием пользователей.

```
~ # ssh-keygen
No user exists for uid 0
~ # ssh
No user exists for uid 0
~ #
```

Рисунок 21 — выполнение команды ssh

Для этого того, чтобы исправить данную ошибку необходимо в папке etc создать файлы passwd и shadow.

```
~ # cd etc
/etc # vi passwd
/etc # vi shadow
/etc # cat passwd
root:x:0:0:root:/root:/bin/ash
/etc # cat shadow
root:!:20000:0:99999:7:::
/etc #
```

Рисунок 22 — содержимое файлов passwd и shadow

Теперь если запустить команду ssh всё проходит без ошибок.

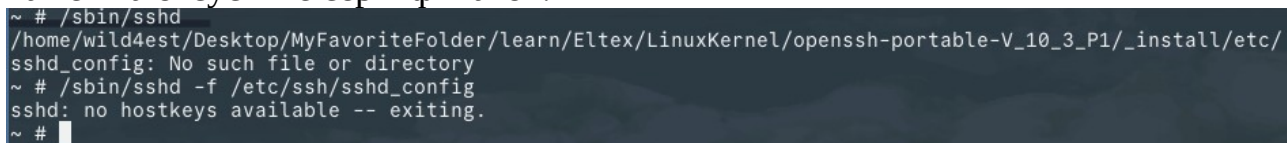
```
~ # ssh
[ 144.927127] random: crng init done
usage: ssh [-46AaCfGgKkMnQsTtVvXxYy] [-B bind_interface] [-b bind_address]
        [-c cipher_spec] [-D [bind_address:]port] [-E log_file]
        [-e escape_char] [-F configfile] [-I pkcs11] [-i identity_file]
        [-J destination] [-L address] [-l login_name] [-m mac_spec]
        [-O ctl_cmd] [-o option] [-P tag] [-p port] [-R address]
        [-S ctl_path] [-W host:port] [-w local_tun[:remote_tun]]
        destination [command [argument ...]]
ssh [-Q query_option]
~ #
```

Рисунок 23 — результат работы команды ssh

```
~ # mkdir .ssh
~ # ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (/root/.ssh/id_ed25519): .ssh/id_1234
Enter passphrase for ".ssh/id_1234" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in .ssh/id_1234
Your public key has been saved in .ssh/id_1234.pub
The key fingerprint is:
SHA256:3+AJJ0bKDhKvsbmKwagTZ7IhbHPnd0a4RuPy2axcmAg root@(none)
The key's randomart image is:
+--[ED25519 256]--+
|
|  . o . o
| . oEo o S o
|B+oB+.+=oB +
| =0* o.+o+. +
| =. . o.=+o
| +o.      =+o
+-----[SHA256]-----+
```

Рисунок 24 — результат работы ssh-keygen

Во время запуска sshd возникает ошибка, связанная с тем, что демон не может найти файлы конфигов. Для того, чтобы исправить эту проблему достаточно просто через флаг -f указать путь до конфигов. В таком случае sshd укажет на отсутствие сертификатов.

A terminal window with a dark background and light-colored text. The text shows a user running the command `/sbin/sshd` from a directory `/home/wild4est/Desktop/MyFavoriteFolder/learn/Eltex/LinuxKernel/openssh-portable-V_10_3_P1/_install/etc/`. The output shows an error: `sshd_config: No such file or directory`. Then the user runs `/sbin/sshd -f /etc/ssh/sshd_config`, and the output is `sshd: no hostkeys available -- exiting.`. The prompt `~ #` is visible at the end of each line.

```
~ # /sbin/sshd
/home/wild4est/Desktop/MyFavoriteFolder/learn/Eltex/LinuxKernel/openssh-portable-V_10_3_P1/_install/etc/
sshd_config: No such file or directory
~ # /sbin/sshd -f /etc/ssh/sshd_config
sshd: no hostkeys available -- exiting.
~ #
```

Рисунок 25 — результат запуска демона sshd.